Intelligent Packaging IoT Sensor Data Analysis

```
#Importing libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
#Loading the dataset
df = pd.read_csv("intelligent_packaging_iot_sensor_data.csv")
```

```
#checking first few records of the data
df.head(5)
```

	Sample_ID	Timestamp	Sensor_Node_ID	Food_Category	Packaging_Material	Storage_Zone	Package_Size	Transport_Condition	S
0	IPKG_00001	2025-01-01 08:00:00	NODE_TEMP_01	Bakery	Glass Container	Warehouse	Medium	Mild Shock	
1	IPKG_00002	2025-01-01 08:10:00	NODE_HUM_02	Dairy	Glass Container	Distribution Center	Small	High Moisture Exposure	
2	IPKG_00003	2025-01-01 08:20:00	NODE_PRESS_04	Bakery	PET Plastic	Cold Chain Storage	Medium	Mild Shock	
3	IPKG_00004	2025-01-01 08:30:00	NODE_MULTI_05	Canned Food	PET Plastic	Distribution Center	Medium	High Moisture Exposure	
4	IPKG_00005	2025-01-01 08:40:00	NODE_PRESS_04	Fresh Produce	Paperboard	Distribution Center	Large	Stable	

5 rows × 22 columns

```
#Getting the shape of the dataset
df.shape
```

```
(15000, 22)
```

```
#getting datatypes and info about data
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 22 columns):
 #   Column                        Non-Null Count  Dtype
---  ------                        --------------  -----
 0   Sample_ID                     15000 non-null  object
 1   Timestamp                     15000 non-null  object
 2   Sensor_Node_ID                15000 non-null  object
 3   Food_Category                 15000 non-null  object
 4   Packaging_Material            15000 non-null  object
 5   Storage_Zone                  15000 non-null  object
 6   Package_Size                  15000 non-null  object
 7   Transport_Condition           15000 non-null  object
 8   Surface_Integrity_Status      15000 non-null  object
 9   Temperature_C                 15000 non-null  float64
 10  Humidity_Percent              15000 non-null  float64
 11  Oxygen_Concentration_Percent  15000 non-null  float64
 12  Carbon_Dioxide_Percent        15000 non-null  float64
 13  Volatile_Gas_ppm              15000 non-null  float64
 14  Seal_Pressure_kPa             15000 non-null  float64
 15  Vibration_g                   15000 non-null  float64
 16  Light_Exposure_lux            15000 non-null  float64
 17  Shelf_Life_Days               15000 non-null  int64
 18  Package_Age_Hours             15000 non-null  int64
 19  Digital_Twin_State            15000 non-null  object
 20  Risk_Score                    15000 non-null  float64
 21  Food_Safety_Risk_Level        15000 non-null  object
```

```
dtypes: float64(9), int64(2), object(11)
memory usage: 2.5+ MB
```

```
#getting all the columns
df.columns
```

```
Index(['Sample_ID', 'Timestamp', 'Sensor_Node_ID', 'Food_Category',
       'Packaging_Material', 'Storage_Zone', 'Package_Size',
       'Transport_Condition', 'Surface_Integrity_Status', 'Temperature_C',
       'Humidity_Percent', 'Oxygen_Concentration_Percent',
       'Carbon_Dioxide_Percent', 'Volatile_Gas_ppm', 'Seal_Pressure_kPa',
       'Vibration_g', 'Light_Exposure_lux', 'Shelf_Life_Days',
       'Package_Age_Hours', 'Digital_Twin_State', 'Risk_Score',
       'Food_Safety_Risk_Level'],
      dtype='object')
```

```
#Checking missing values if any
df.isnull().sum()
```

	0
Sample_ID	0
Timestamp	0
Sensor_Node_ID	0
Food_Category	0
Packaging_Material	0
Storage_Zone	0
Package_Size	0
Transport_Condition	0
Surface_Integrity_Status	0
Temperature_C	0
Humidity_Percent	0
Oxygen_Concentration_Percent	0
Carbon_Dioxide_Percent	0
Volatile_Gas_ppm	0
Seal_Pressure_kPa	0
Vibration_g	0
Light_Exposure_lux	0
Shelf_Life_Days	0
Package_Age_Hours	0
Digital_Twin_State	0
Risk_Score	0
Food_Safety_Risk_Level	0

dtype: int64

```
#checking duplicates if any in the dataset
df.duplicated().sum()
```

```
np.int64(0)
```

```
#Checking datatypes of all columns
df.dtypes
```

	0
Sample_ID	object
Timestamp	object
Sensor_Node_ID	object
Food_Category	object
Packaging_Material	object
Storage_Zone	object
Package_Size	object
Transport_Condition	object
Surface_Integrity_Status	object
Temperature_C	float64
Humidity_Percent	float64
Oxygen_Concentration_Percent	float64
Carbon_Dioxide_Percent	float64
Volatile_Gas_ppm	float64
Seal_Pressure_kPa	float64
Vibration_g	float64
Light_Exposure_lux	float64
Shelf_Life_Days	int64
Package_Age_Hours	int64
Digital_Twin_State	object
Risk_Score	float64
Food_Safety_Risk_Level	object

dtype: object

```
#encountered that timestamp is in object format we need to change into datetime format
df['Timestamp'] = pd.to_datetime(df['Timestamp'])

print(df['Timestamp'].dtype)
```

```
datetime64[ns]
```

```
df.dtypes
```

	0
Sample_ID	object
Timestamp	datetime64[ns]
Sensor_Node_ID	object
Food_Category	object
Packaging_Material	object
Storage_Zone	object
Package_Size	object
Transport_Condition	object
Surface_Integrity_Status	object
Temperature_C	float64
Humidity_Percent	float64
Oxygen_Concentration_Percent	float64
Carbon_Dioxide_Percent	float64
Volatile_Gas_ppm	float64
Seal_Pressure_kPa	float64
Vibration_g	float64
Light_Exposure_lux	float64
Shelf_Life_Days	int64
Package_Age_Hours	int64
Digital_Twin_State	object
Risk_Score	float64
Food_Safety_Risk_Level	object

dtype: object

```
#Final checks about the data
print("Missing values:", df.isnull().sum().sum())
print("Duplicate rows:", df.duplicated().sum())
print("Duplicate Sample IDs:", df['Sample_ID'].duplicated().sum())
```

```
Missing values: 0
Duplicate rows: 0
Duplicate Sample IDs: 0
```

```
#Creating a duplicate datset so that our real dataset wont get any changes
df_clean = df.copy()
```

EXPLRATORY DATA ANALYSIS

```
#1. Food Categories
df_clean['Food_Category'].value_counts()
```

Food_Category	count
Dairy	1560
Bakery	1536
Fresh Produce	1515
Meat Product	1513
Canned Food	1507
Beverage	1503
Frozen Food	1483
Snack Food	1478
Seafood	1468
Ready Meal	1437

dtype: int64

```
#2.Risk levels'
df_clean['Food_Safety_Risk_Level'].value_counts()
```

Food_Safety_Risk_Level	count
Critical Risk	5221
Medium Risk	4100
High Risk	2987
Low Risk	2692

dtype: int64

```
#3 Packaging Materials
df_clean['Packaging_Material'].value_counts()
```

Packaging_Material	count
PET Plastic	2210
Glass Container	2196
Paperboard	2156
Biodegradable Film	2128
Composite Film	2123
Aluminum Laminate	2105
Vacuum Pouch	2082

dtype: int64

```
#4 Surface Condiations
df_clean['Surface_Integrity_Status'].value_counts()
```

Surface_Integrity_Status	count
Flawless Package	6792
Seal Defect	3511
Surface Deformation	2881
Contamination Defect	1816

dtype: int64

```
Start coding or generate with AI.
```

Top 20 questions on our dataset for data analysis

Q1. How many packaging samples are there in total?

```
total_samples = df_clean.shape[0]

print("Total samples:", total_samples)
```

```
Total samples: 15000
```

Q2. Which food categories have the most samples?

```
df_clean['Food_Category'].value_counts().head(3)
```

Food_Category	count
Dairy	1560
Bakery	1536
Fresh Produce	1515

dtype: int64

Q3. Which food category has the highest average risk score?

```
avg_risk_food = (
    df_clean.groupby('Food_Category')['Risk_Score']
    .mean()
    .sort_values(ascending=False)
)

print(avg_risk_food)
```

```
Food_Category
Seafood          0.611926
Frozen Food      0.605748
Bakery           0.600657
Beverage         0.597879
Fresh Produce    0.595504
Dairy            0.594388
Meat Product     0.593515
Ready Meal       0.592625
Canned Food      0.586884
Snack Food       0.572244
Name: Risk_Score, dtype: float64
```

Q4.Which packaging material has the highest average risk?

```
df_clean.groupby('Packaging_Material')['Risk_Score'].mean().sort_values(ascending=False)
```

Packaging_Material	Risk_Score
Paperboard	0.605122
PET Plastic	0.597968
Glass Container	0.594732
Vacuum Pouch	0.594680
Composite Film	0.593939
Aluminum Laminate	0.590429
Biodegradable Film	0.588813

dtype: float64

Q5.How many samples fall into each food safety risk level?

```
risk_counts = df_clean['Food_Safety_Risk_Level'].value_counts()

print(risk_counts)
```

```
Food_Safety_Risk_Level
Critical Risk    5221
Medium Risk      4100
High Risk        2987
Low Risk         2692
Name: count, dtype: int64
```

Q6. Which package condition occurs most often?

```
df_clean['Surface_Integrity_Status'].value_counts()
```

Surface_Integrity_Status	count
Flawless Package	6792
Seal Defect	3511
Surface Deformation	2881
Contamination Defect	1816

dtype: int64

Q7.Does package condition affect the average risk score?

```
df_clean.groupby('Surface_Integrity_Status')['Risk_Score'].mean().sort_values(ascending=False)
```

Surface_Integrity_Status	Risk_Score
Contamination Defect	1.000000
Seal Defect	0.842514
Surface Deformation	0.711749
Flawless Package	0.309551

dtype: float64

Q8.Which storage zone has the highest average risk?

```
df_clean.groupby('Storage_Zone')['Risk_Score'].mean().sort_values(ascending=False)
```

Storage_Zone	Risk_Score
Cold Chain Storage	0.600220
Transport Vehicle	0.598642
Retail Shelf	0.595690
Distribution Center	0.591522
Warehouse	0.589619

dtype: float64

Q9.Which transport condition has the highest average risk?

```
df_clean.groupby('Transport_Condition')['Risk_Score'].mean().sort_values(ascending=False)
```

Transport_Condition	Risk_Score
Mild Shock	0.598698
Frequent Vibration	0.596815
High Moisture Exposure	0.596303
Temperature Fluctuation	0.595180
Stable	0.588692

dtype: float64

Q10.Does package size make a difference to risk?

```
df_clean.groupby('Package_Size')['Risk_Score'].mean().sort_values(ascending=False)
```

Package_Size	Risk_Score
Small	0.600487
Bulk	0.594745
Medium	0.594341
Large	0.590833

dtype: float64

Q11.Which food category has the highest average temperature?

```
df_clean.groupby('Food_Category')['Temperature_C'].mean().sort_values(ascending=False)
```

Food_Category	Temperature_C
Seafood	6.978304
Beverage	6.877039
Canned Food	6.848182
Ready Meal	6.838170
Frozen Food	6.834889
Fresh Produce	6.834587
Bakery	6.834199
Dairy	6.781513
Meat Product	6.736490
Snack Food	6.611015

dtype: float64

Q13.Which food category has the highest average humidity?

```
df_clean.groupby('Food_Category')['Humidity_Percent'].mean().sort_values(ascending=False)
```

Food_Category	Humidity_Percent
Bakery	62.697454
Seafood	62.494905
Beverage	62.470572
Fresh Produce	62.452112
Ready Meal	62.297091
Frozen Food	62.245745
Dairy	62.183077
Meat Product	62.135962
Canned Food	62.095720
Snack Food	61.622639

dtype: float64

Q14. Which food category has the longest average shelf life?

```
df_clean.groupby('Food_Category')['Shelf_Life_Days'].mean().sort_values(ascending=False)
```

Food_Category	Shelf_Life_Days
Ready Meal	11.915101
Snack Food	11.729364
Dairy	11.647436
Seafood	11.589237
Meat Product	11.526107
Frozen Food	11.507755
Canned Food	11.506304
Bakery	11.490234
Fresh Produce	11.476568
Beverage	11.302063

dtype: float64

Q15.Which packaging material has the lowest average seal pressure?

```
df_clean.groupby('Packaging_Material')['Seal_Pressure_kPa'].mean().sort_values()
```

Packaging_Material	Seal_Pressure_kPa
Paperboard	78.728210
PET Plastic	79.097661
Vacuum Pouch	79.218199
Composite Film	79.313782
Aluminum Laminate	79.446822
Glass Container	79.535337
Biodegradable Film	79.723792

dtype: float64

Q16. Which transport condition has the highest average vibration?

```
df_clean.groupby('Transport_Condition')['Vibration_g'].mean().sort_values(ascending=False)
```

Transport_Condition	Vibration_g
Frequent Vibration	0.318220
High Moisture Exposure	0.318013
Temperature Fluctuation	0.316483
Mild Shock	0.314716
Stable	0.311422

dtype: float64

Q17. Which storage zone has the highest average light exposure?

```
df_clean.groupby('Storage_Zone')['Light_Exposure_lux'].mean().sort_values(ascending=False)
```

Storage_Zone	Light_Exposure_lux
Cold Chain Storage	227.151490
Warehouse	226.470150
Retail Shelf	226.231750
Transport Vehicle	226.079152
Distribution Center	224.941586

dtype: float64

q19. Which food category has the highest percentage of critical-risk samples?

```
critical_percentage = (
    df_clean.groupby('Food_Category')['Food_Safety_Risk_Level']
    .apply(lambda x: (x == 'Critical Risk').mean() * 100)
    .sort_values(ascending=False)
)

print(critical_percentage)
```

```
Food_Category
Seafood          37.942779
Beverage         35.728543
Frozen Food      35.670937
Fresh Produce    35.511551
Ready Meal       35.421016
Dairy            34.871795
Bakery           34.570312
Canned Food      34.173855
Meat Product     33.906147
Snack Food       30.311231
Name: Food_Safety_Risk_Level, dtype: float64
```

Q20. Which packaging material has the highest percentage of high or critical risk samples?

```
high_risk_percentage = (
    df_clean.groupby('Packaging_Material')['Food_Safety_Risk_Level']
    .apply(lambda x: x.isin(['High Risk', 'Critical Risk']).mean() * 100)
    .sort_values(ascending=False)
)

print(high_risk_percentage)
```

```
Packaging_Material
Paperboard            56.539889
PET Plastic           55.475113
Vacuum Pouch          54.947166
Glass Container       54.690346
Composite Film        53.838907
```

```
Aluminum Laminate     53.776722
Biodegradable Film    53.712406
Name: Food_Safety_Risk_Level, dtype: float64
```

Start coding or generate with AI.

DATA VISUALIZATION

```
#Visualization 1 – Food Safety Risk Distribution

risk_counts = df_clean['Food_Safety_Risk_Level'].value_counts()

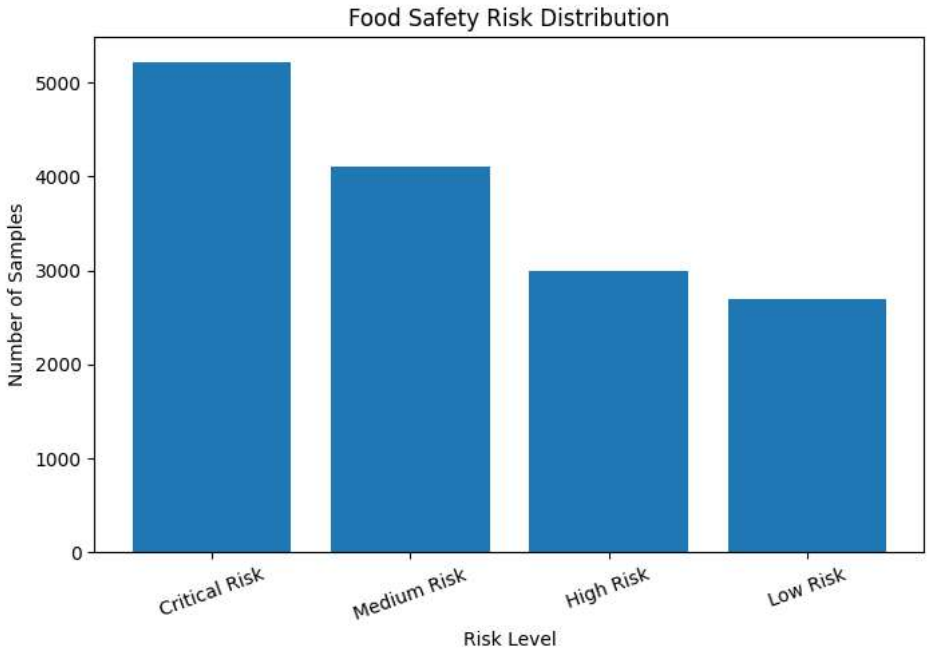
plt.figure(figsize=(8,5))

plt.bar(risk_counts.index, risk_counts.values)

plt.title('Food Safety Risk Distribution')
plt.xlabel('Risk Level')
plt.ylabel('Number of Samples')

plt.xticks(rotation=20)

plt.show()
```




```
#Visualization 2: Average Risk by Package Condition

risk_by_condition = (
    df_clean.groupby('Surface_Integrity_Status')['Risk_Score']
    .mean()
    .sort_values(ascending=False)
)

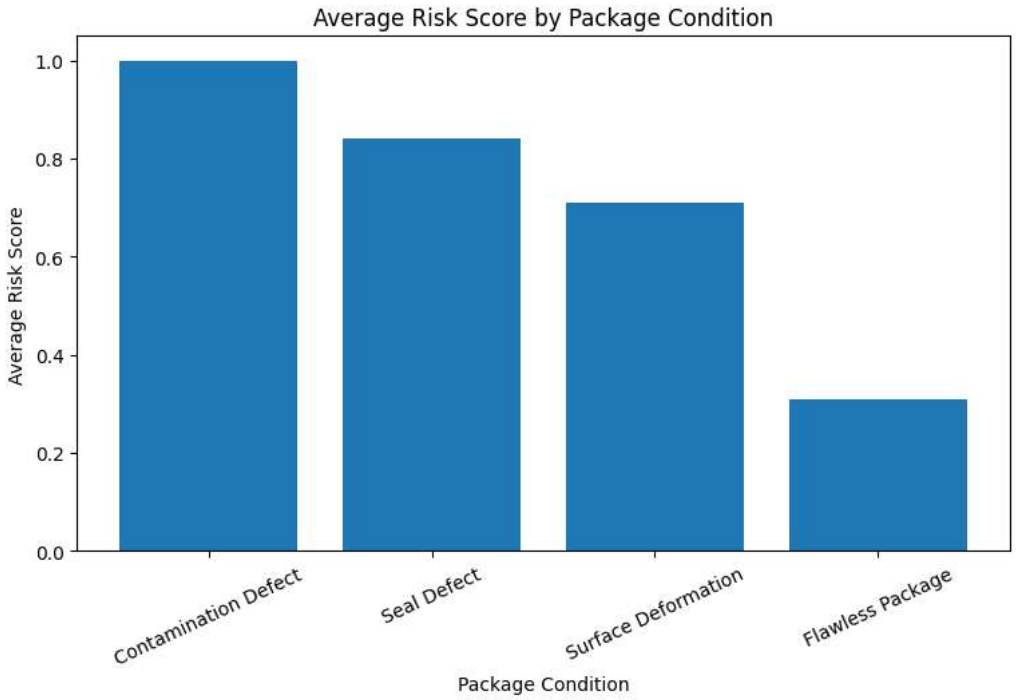
plt.figure(figsize=(9,5))

plt.bar(
    risk_by_condition.index,
    risk_by_condition.values
)

plt.title('Average Risk Score by Package Condition')
plt.xlabel('Package Condition')
plt.ylabel('Average Risk Score')

plt.xticks(rotation=25)

plt.show()
```




```
#Visualization 3 : Average Risk by Food Category

food_risk = (
    df_clean.groupby('Food_Category')['Risk_Score']
    .mean()
    .sort_values(ascending=False)
)

plt.figure(figsize=(10,5))

plt.bar(food_risk.index, food_risk.values)

plt.title('Average Risk Score by Food Category')
plt.xlabel('Food Category')
plt.ylabel('Average Risk Score')

plt.xticks(rotation=45)

plt.show()
```

Average Risk Score by Food Category

```
#Visualization 4 - Packaging Material vs Risk

material_risk = (
    df_clean.groupby('Packaging_Material')['Risk_Score']
    .mean()
```